

# authapp package

## Subpackages

- authapp.migrations package
  - Submodules
  - [authapp.migrations.0001\\_initial](#) module
    - `Migration`
      - `Migration.dependencies`
      - `Migration.initial`
      - `Migration.operations`
  - [authapp.migrations.0002\\_alter\\_morphologicalproperties\\_boccode\\_and\\_more](#) module
    - `Migration`
      - `Migration.dependencies`
      - `Migration.operations`
  - [authapp.migrations.0003\\_profile\\_assigned\\_to\\_assignmenthistory](#) module
    - `Migration`
      - `Migration.dependencies`
      - `Migration.operations`
  - [authapp.migrations.0004\\_surveydata\\_point\\_assignment](#) module
    - `Migration`
      - `Migration.dependencies`
      - `Migration.operations`
  - [authapp.migrations.0005\\_alter\\_surveydata\\_point\\_assignment](#) module
    - `Migration`
      - `Migration.dependencies`
      - `Migration.operations`
  - [authapp.migrations.0006\\_remove\\_sitecharacteristics\\_tehsil\\_and\\_more](#) module
    - `Migration`
      - `Migration.dependencies`

- `Migration.operations`
- `authapp.migrations.0007_labhorizondata_bulk_density_labhorizondata_gravel` module
  - `Migration`
    - `Migration.dependencies`
    - `Migration.operations`
- `authapp.migrations.0008_rename_bulk_density_labhorizondata_bulkdensity` module
  - `Migration`
    - `Migration.dependencies`
    - `Migration.operations`
- [Module contents](#)

## Submodules

### authapp.admin module

### authapp.apps module

---

**class** `authapp.apps.AuthappConfig(app_name, app_module)` [\[source\]](#)

Bases: `AppConfig`

`default_auto_field= 'django.db.models.BigAutoField'`

`name= 'authapp'`

### authapp.authentication module

---

**class** `authapp.authentication.BlockCheckingJWTAuthentication(*args, **kwargs)` [\[source\]](#)

Bases: `JWTAuthentication`

Extends SimpleJWT's `JWTAuthentication` to:

- reject tokens if user profile `ACTIVE_STATUS` is Blocked/Inactive
- optionally check `token_version` claim vs `profile.token_version` for instant revocation

**`authenticate(request)`** [\[source\]](#)

Authenticate the request and return a two-tuple of (user, token).

# authapp.middleware module

---

**class** `authapp.middleware.JWTAuthCookieMiddleware`(`get_response`) [\[source\]](#)

Bases: `object`

Middleware that injects JWT access tokens from cookies into request headers.

This allows authentication mechanisms (e.g., Django REST Framework JWT) that expect an *Authorization* header to work seamlessly when the token is stored in an HTTP cookie instead.

# authapp.models module

---

**class** `authapp.models.AssignmentHistory`(`*args, **kwargs`) [\[source\]](#)

Bases: `Model`

Tracks assignment changes for a user profile.

Each record represents a transition where a profile was reassigned from one user to another. This provides an audit trail for assignment changes over time.

**exception** `DoesNotExist`

Bases: `ObjectDoesNotExist`

**exception** `MultipleObjectsReturned`

Bases: `MultipleObjectsReturned`

**changed\_at**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**from\_user**

Accessor to the related object on the forward side of a many-to-one or one-to-one (via `ForwardOneToOneDescriptor` subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

**from\_user\_id**

```
get_next_by_changed_at(*, field=<django.db.models.fields.DateTimeField: changed_at>,
is_next=True, **kwargs)
```

```
get_previous_by_changed_at(*, field=<django.db.models.fields.DateTimeField: changed_at>,
is_next=False, **kwargs)
```

## id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**objects=** <django.db.models.manager.Manager object>

## profile

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

## profile\_id

## to\_user

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

## to\_user\_id

---

```
class authapp.models.Jurisdiction(*args, **kwargs) \[source\]
```

Bases: `Model`

Represents an administrative or legal jurisdiction.

### **assigned\_district**

Unique name or identifier of the district.

**Type:** str

### **boundary**

Geographic boundary of the jurisdiction.

**Type:** PolygonField

### *exception* **DoesNotExist**

Bases: `ObjectDoesNotExist`

### *exception* **MultipleObjectsReturned**

Bases: `MultipleObjectsReturned`

### **assigned\_district**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

### **boundary**

### **id**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

### **labassignment\_set**

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

**objects=** <django.db.models.manager.Manager object>

## pointassignment\_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

## pointreservation\_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

## users

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

---

`class authapp.models.LabAssignment(*args, **kwargs)` [\[source\]](#)

Bases: `Model`

Tracks assignment of survey data to laboratory analysts.

Each record represents the assignment of a single SOSAM survey to a LAB Analyst for laboratory processing, including who made the assignment, when it occurred, and the applicable jurisdiction.

**exception DoesNotExist**

Bases: `ObjectDoesNotExist`

**exception MultipleObjectsReturned**

Bases: `MultipleObjectsReturned`

**assigned\_at**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**assigned\_by**

Accessor to the related object on the forward side of a many-to-one or one-to-one (via `ForwardOneToOneDescriptor` subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

**assigned\_by\_id**

**assigned\_to**

Accessor to the related object on the forward side of a many-to-one or one-to-one (via `ForwardOneToOneDescriptor` subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

**assigned\_to\_id**

```
get_next_by_assigned_at(*, field=<django.db.models.fields.DateTimeField: assigned_at>,
is_next=True, **kwargs)
```

```
get_previous_by_assigned_at(*, field=<django.db.models.fields.DateTimeField: assigned_at>,
is_next=False, **kwargs)
```

**id**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**jurisdiction**

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

**jurisdiction\_id**

**lab\_survey\_data**

Accessor to the related object on the reverse side of a one-to-one relation.

In the example:

```
class Restaurant(Model):
    place = OneToOneField(Place, related_name='restaurant')
```

`Place.restaurant` is a `ReverseOneToOneDescriptor` instance.

**objects=** *<django.db.models.manager.Manager object>*

**sosam\_code**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.



## survey\_data

Accessor to the related object on the forward side of a one-to-one relation.

In the example:

```
class Restaurant(Model):  
    place = OneToOneField(Place, related_name='restaurant')
```

`Restaurant.place` is a `ForwardOneToOneDescriptor` instance.

## survey\_data\_id

---

`class authapp.models.LabHorizonData(*args, **kwargs)` [\[source\]](#)

Bases: `Model`

Stores laboratory analysis results for a single soil horizon.

Each record represents physical and chemical properties measured in the laboratory for one horizon of a SOSAM survey profile.

### *exception* DoesNotExist

Bases: `ObjectDoesNotExist`

### *exception* MultipleObjectsReturned

Bases: `MultipleObjectsReturned`

## availFe

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## availK

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## availN

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## availP

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **bulkdensity**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **caco3**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **cec**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **clay**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **created\_at**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **ec**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **esp**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **exA1**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **exCatCa**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **exCatK**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## **exCatMg**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## **exCatNa**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## **exH**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

```
get_next_by_created_at(* , field=<django.db.models.fields.DateTimeField: created_at>,  
is_next=True, **kwargs)
```

```
get_next_by_updated_at(* , field=<django.db.models.fields.DateTimeField: updated_at>,  
is_next=True, **kwargs)
```

```
get_previous_by_created_at(* , field=<django.db.models.fields.DateTimeField: created_at>,  
is_next=False, **kwargs)
```

```
get_previous_by_updated_at(* , field=<django.db.models.fields.DateTimeField: updated_at>,  
is_next=False, **kwargs)
```

## **gravel**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## **horizon\_code**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## **id**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## **lab\_survey**

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):  
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

**lab\_survey\_id**

**objects=** <django.db.models.manager.Manager object>

**ogc**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**ph**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**sand**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**silt**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**sosam\_code**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**totalB**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**totalCu**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**totalMn**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

### **totalzn**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

### **updated\_at**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

---

**class** `authapp.models.LabSurveyData(*args, **kwargs)` [\[source\]](#)

Bases: `Model`

Stores laboratory survey data uploaded for assigned SOSAM surveys.

This model tracks the receipt and upload of laboratory analysis data, who uploaded it, timestamps, and provides helpers to evaluate horizon-level completion status.

### **exception DoesNotExist**

Bases: `ObjectDoesNotExist`

### **exception MultipleObjectsReturned**

Bases: `MultipleObjectsReturned`

**get\_completed\_horizons()** [\[source\]](#)

**get\_next\_by\_updated\_at**(\**field*=<django.db.models.fields.DateTimeField: updated\_at>, *is\_next*=True, \*\*kwargs)

**get\_next\_by\_uploaded\_at**(\**field*=<django.db.models.fields.DateTimeField: uploaded\_at>, *is\_next*=True, \*\*kwargs)

**get\_previous\_by\_updated\_at**(\**field*=<django.db.models.fields.DateTimeField: updated\_at>, *is\_next*=False, \*\*kwargs)

**get\_previous\_by\_uploaded\_at**(\**field*=<django.db.models.fields.DateTimeField: uploaded\_at>, *is\_next*=False, \*\*kwargs)

**get\_total\_horizons()** [\[source\]](#)

**id**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**is\_all\_horizons\_complete()** [\[source\]](#)

**lab\_assignment**

Accessor to the related object on the forward side of a one-to-one relation.

In the example:

```
class Restaurant(Model):
    place = OneToOneField(Place, related_name='restaurant')
```

`Restaurant.place` is a `ForwardOneToOneDescriptor` instance.

**lab\_assignment\_id**

**lab\_horizons**

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

**objects=** *<django.db.models.manager.Manager object>*

**sosam\_code**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**updated\_at**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## uploaded\_at

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## uploader

Accessor to the related object on the forward side of a many-to-one or one-to-one (via `ForwardOneToOneDescriptor` subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

## uploader\_id

---

`class authapp.models.MorphologicalProperties(*args, **kwargs)` [\[source\]](#)

Bases: `Model`

Stores morphological characteristics of soil horizons for a given survey.

This model captures horizon-specific codes, color, texture, structure, and other morphological properties for each soil profile associated with a survey point. Each record corresponds to a single horizon of a profile.

## exception DoesNotExist

Bases: `ObjectDoesNotExist`

## exception MultipleObjectsReturned

Bases: `MultipleObjectsReturned`

## bocode

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## bodcode

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## botcode

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **cfacode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **cfncode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **cfscode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **codcode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **coecode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **color**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **comcode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **coqesize**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **coscode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **cowcode**



A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **crkdepth**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **crknature**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **crkwidth**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **cuccode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **cuocode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **cutcode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **cuycode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **depthcode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **effcode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **horizon\_code**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **id**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **moacode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **mosccode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **moscode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **objects= <django.db.models.manager.Manager object>**

#### **poqcode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **poscode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **roqcode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **roscode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **sddd000000**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## **slicode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## **stgcode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## **stscore**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## **sttcode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## **survey\_data**

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):  
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

## **survey\_data\_id**

## **texcode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

---

`class authapp.models.PointAssignment(*args, **kwargs)` [\[source\]](#)

Bases: `Model`

Represents the assignment of a SOSAM point from one user to another.

This model records who assigned the SOSAM code, who received it, when the assignment occurred, and the jurisdiction associated with the point.

## **exception DoesNotExist**

Bases: `ObjectDoesNotExist`

## **exception `MultipleObjectsReturned`**

Bases: `MultipleObjectsReturned`

## **`assigned_at`**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## **`assigned_by`**

Accessor to the related object on the forward side of a many-to-one or one-to-one (via `ForwardOneToOneDescriptor` subclass) relation.

In the example:

```
class Child(Model):  
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

## **`assigned_by_id`**

## **`assigned_to`**

Accessor to the related object on the forward side of a many-to-one or one-to-one (via `ForwardOneToOneDescriptor` subclass) relation.

In the example:

```
class Child(Model):  
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

## **`assigned_to_id`**

`get_next_by_assigned_at(*, field=<django.db.models.fields.DateTimeField: assigned_at>, is_next=True, **kwargs)`

`get_previous_by_assigned_at(*, field=<django.db.models.fields.DateTimeField: assigned_at>, is_next=False, **kwargs)`

## id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## jurisdiction

Accessor to the related object on the forward side of a many-to-one or one-to-one (via `ForwardOneToOneDescriptor` subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

## jurisdiction\_id

`objects=` <django.db.models.manager.Manager object>

## sosam\_code

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## survey\_data

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

---

```
class authapp.models.PointReservation(*args, **kwargs) \[source\]
```

Bases: `Model`

Represents a reservation of a SOSAM point by a user.

This model tracks which user reserved a specific SOSAM code, when the reservation occurred, and the jurisdiction in which the reservation applies. The reserving user's username is denormalized for quick access and historical accuracy.

**exception DoesNotExist**

Bases: `ObjectDoesNotExist`

**exception MultipleObjectsReturned**

Bases: `MultipleObjectsReturned`

**get\_next\_by\_reserved\_at**(\* , field=<django.db.models.fields.DateTimeField: reserved\_at>, is\_next=True, \*\*kwargs)

**get\_previous\_by\_reserved\_at**(\* , field=<django.db.models.fields.DateTimeField: reserved\_at>, is\_next=False, \*\*kwargs)

**id**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**jurisdiction**

Accessor to the related object on the forward side of a many-to-one or one-to-one (via `ForwardOneToOneDescriptor` subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

**jurisdiction\_id**

**objects**= <django.db.models.manager.Manager object>

**reserved\_at**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**reserved\_by**

Accessor to the related object on the forward side of a many-to-one or one-to-one (via `ForwardOneToOneDescriptor` subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

## reserved\_by\_id

## reserved\_by\_username

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## save(\*args, \*\*kwargs) [\[source\]](#)

Save the current instance. Override this in a subclass if you want to control the saving process.

The ‘force\_insert’ and ‘force\_update’ parameters can be used to insist that the “save” must be an SQL insert or update (or equivalent for non-SQL backends), respectively. Normally, they should not be set.

## sosam\_code

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

---

```
class authapp.models.Profile(*args, **kwargs) \[source\]
```

Bases: `Model`

Extends the default Django User model with domain-specific profile data.

This model stores user status, designation, contact information, location, assignment relationships, and lifecycle timestamps such as block, unblock, freeze, and unfreeze events.

## Business Rules:

- A user can have only one profile.
- A designation is allowed only if the user does not belong to any group.
- Status transitions automatically update relevant timestamps.
- Assignment changes are logged in AssignmentHistory.

## ACTIVE\_STATUS

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## CREATION\_DATE

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**DESIGNATION\_CHOICES=** *[('Field Surveyor', 'Field Surveyor'), ('LAB Analyst', 'LAB Analyst')]*

*exception* **DoesNotExist**

Bases: `ObjectDoesNotExist`

*exception* **MultipleObjectsReturned**

Bases: `MultipleObjectsReturned`

**STATUS\_CHOICES=** *[('Active', 'Active'), ('Inactive', 'Inactive'), ('Blocked', 'Blocked'), ('Frozen', 'Frozen')]*

## assigned\_to

Accessor to the related object on the forward side of a many-to-one or one-to-one (via `ForwardOneToOneDescriptor` subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

## assigned\_to\_id

## assignment\_history

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```



`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

### **blocked\_at**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

### **clean()** [\[source\]](#)

Hook for doing any extra model-wide validation after `clean()` has been called on every field by `self.clean_fields`. Any `ValidationError` raised by this method will not be associated with a particular field; it will have a special-case association with the field defined by `NON_FIELD_ERRORS`.

### **created\_by**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

### **department**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

### **designation**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

### **district**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

### **email**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

### **frozen\_at**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

### **get\_ACTIVE\_STATUS\_display(\*, field=<django.db.models.fields.CharField: ACTIVE\_STATUS>)**

**get\_designation\_display**(\*, field=<django.db.models.fields.CharField: designation>)

**get\_next\_by\_CREATION\_DATE**(\*, field=<django.db.models.fields.DateTimeField: CREATION\_DATE>, is\_next=True, \*\*kwargs)

**get\_previous\_by\_CREATION\_DATE**(\*, field=<django.db.models.fields.DateTimeField: CREATION\_DATE>, is\_next=False, \*\*kwargs)

**id**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**mobile\_no**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**objects**= <django.db.models.manager.Manager object>

*property* **role**

**save**(\*args, \*\*kwargs) [\[source\]](#)

Save the current instance. Override this in a subclass if you want to control the saving process.

The 'force\_insert' and 'force\_update' parameters can be used to insist that the "save" must be an SQL insert or update (or equivalent for non-SQL backends), respectively. Normally, they should not be set.

**state**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**unblocked\_at**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**unfrozen\_at**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## user

Accessor to the related object on the forward side of a one-to-one relation.

In the example:

```
class Restaurant(Model):  
    place = OneToOneField(Place, related_name='restaurant')
```

`Restaurant.place` is a `ForwardOneToOneDescriptor` instance.

## user\_id

---

**class** `authapp.models.SiteCharacteristics(*args, **kwargs)` [\[source\]](#)

Bases: `Model`

Stores site-level physical, geographic, and land-use characteristics associated with a field survey.

This model captures common project metadata, geographic coordinates, map-derived parameters, landform and soil attributes, cropping patterns, and environmental constraints for a specific SOSAM survey point.

## exception `DoesNotExist`

Bases: `ObjectDoesNotExist`

## exception `MultipleObjectsReturned`

Bases: `MultipleObjectsReturned`

## accuracy

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## crop\_kharif

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## crop\_rabi

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## crop\_summer

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **date\_time**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **district**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **elevation**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **erocode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **forest\_proximity**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **geology**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **id**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **investigator**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **iplucode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **ipluuqcode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **land\_form**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **land\_use**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **land\_use\_map**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **lat**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **lo\_series**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **long**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **lso\_depth**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **lso\_drain**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **lso\_erosion**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **lso\_landcap**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **lso\_taxo**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **lso\_temprg**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **lso\_texture**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **nodal\_officer**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **objects= <django.db.models.manager.Manager object>**

#### **pancode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **percode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **profile\_type**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **project\_name**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **saltcode**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **sddd000000**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **sf\_gravelly**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **sf\_rocky**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **sf\_stony**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **slope**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **slope\_status**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **soil\_constraints**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **soil\_name**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **soil\_temperature**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

#### **surcondico**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## survey\_data

Accessor to the related object on the forward side of a one-to-one relation.

In the example:

```
class Restaurant(Model):  
    place = OneToOneField(Place, related_name='restaurant')
```

`Restaurant.place` is a `ForwardOneToOneDescriptor` instance.

## survey\_data\_id

## surveyor

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## topocode

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

---

**class** `authapp.models.SurveyData(*args, **kwargs)` [\[source\]](#)

Bases: `Model`

Stores field survey data uploaded for a SOSAM point.

This model tracks survey uploads, their uploader, lifecycle timestamps, linkage to point assignments, and downstream laboratory processing status.

## exception DoesNotExist

Bases: `ObjectDoesNotExist`

## exception MultipleObjectsReturned

Bases: `MultipleObjectsReturned`

`get_next_by_uploaded_at(*, field=<django.db.models.fields.DateTimeField: uploaded_at>, is_next=True, **kwargs)`

`get_previous_by_uploaded_at(*, field=<django.db.models.fields.DateTimeField: uploaded_at>, is_next=False, **kwargs)`



*property* **has\_lab\_assignment**

**horizon\_no**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**id**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**is\_sent\_to\_lab**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**lab\_assignment**

Accessor to the related object on the reverse side of a one-to-one relation.

In the example:

```
class Restaurant(Model):  
    place = OneToOneField(Place, related_name='restaurant')
```

`Place.restaurant` is a `ReverseOneToOneDescriptor` instance.

*property* **lab\_completion\_status**

**lab\_sent\_at**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

**morphological\_properties**

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):  
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

**objects**= *<django.db.models.manager.Manager object>*

### **point\_assignment**

Accessor to the related object on the forward side of a many-to-one or one-to-one (via `ForwardOneToOneDescriptor` subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

### **point\_assignment\_id**

### **site\_characteristics**

Accessor to the related object on the reverse side of a one-to-one relation.

In the example:

```
class Restaurant(Model):
    place = OneToOneField(Place, related_name='restaurant')
```

`Place.restaurant` is a `ReverseOneToOneDescriptor` instance.

### **sosam\_code**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

### **uploaded\_at**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

### **uploader**

Accessor to the related object on the forward side of a many-to-one or one-to-one (via `ForwardOneToOneDescriptor` subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

## uploader\_id

---

`class authapp.models.UserJurisdiction(*args, **kwargs)` [\[source\]](#)

Bases: `Model`

Associates a user with a jurisdiction.

This model enforces a unique relationship between a user and a jurisdiction and generates a derived unique identifier (*email\_jurisdiction*) based on the user's email and the jurisdiction ID.

### Constraints:

- A user cannot be assigned to the same jurisdiction more than once.
- Each email-jurisdiction pair is globally unique.

## exception DoesNotExist

Bases: `ObjectDoesNotExist`

## exception MultipleObjectsReturned

Bases: `MultipleObjectsReturned`

## email\_jurisdiction

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

## jurisdiction

Accessor to the related object on the forward side of a many-to-one or one-to-one (via `ForwardOneToOneDescriptor` subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

**jurisdiction\_id**

**objects=** <django.db.models.manager.Manager object>

**save(\*args, \*\*kwargs)** [\[source\]](#)

Save the current instance. Override this in a subclass if you want to control the saving process.

The ‘force\_insert’ and ‘force\_update’ parameters can be used to insist that the “save” must be an SQL insert or update (or equivalent for non-SQL backends), respectively. Normally, they should not be set.

**user**

Accessor to the related object on the forward side of a many-to-one or one-to-one (via `ForwardOneToOneDescriptor` subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

**user\_id**

## authapp.pointgeojsondata module

---

`class` `authapp.pointgeojsondata.AUAssignedPointsAPIView(**kwargs)` [\[source\]](#)

Bases: `APIView`

API View for AU users to view all assigned points in their jurisdiction.

**Permissions:**

- Only AU users can access

**Query Parameters:**

- `assigned_to`: (Optional) Filter by UU profile ID to see specific user’s assignments

## Response:

List of all point assignments with assigned\_to/assigned\_by details

```
get(request) \[source\]
```

```
permission_classes= [class 'rest\_framework.permissions.IsAuthenticated'>]
```

---

```
class authapp.pointgeojsondata.AUReservedPointsAPIView(**kwargs) \[source\]
```

Bases: `APIView`

API View for retrieving reserved points with assignment details.

### Permissions:

- AU: View own reserved points
- SU: View any user's reserved points (requires user\_id)

### Query Parameters:

- jurisdiction\_name: Filter by jurisdiction name

### Path Parameters:

- user\_id: (SU only) Target user ID
- jurisdiction\_id: (Optional) Filter by jurisdiction ID

## Response:

Includes point assignment details (UU name, status, timestamps)

```
get(request, user_id=None, jurisdiction_id=None) \[source\]
```

```
permission_classes= [class 'rest\_framework.permissions.IsAuthenticated'>]
```

---

```
class authapp.pointgeojsondata.AssignPointAPIView(**kwargs) \[source\]
```

Bases: `APIView`

API View for assigning reserved points to UU users.

### Permissions:

- Only AU users can assign points

### Request Body:

- sosam\_codes: List of SOSAM codes to assign
- assigned\_to: Target UU user ID
- jurisdiction: Jurisdiction ID

## Validation:

- AU must belong to the jurisdiction
- UU must belong to the jurisdiction
- Prevents duplicate assignments

```
permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]
```

```
post(request) \[source\]
```

---

```
class authapp.pointgeojsondata.CompletedSurveysAPIView(**kwargs) \[source\]
```

Bases: `APIView`

API View for retrieving completed surveys with full details.

## Permissions:

- UU: View own surveys only
- AU/SU: View any user's surveys

## Path Parameters:

- user\_id: Target user's profile ID
- jurisdiction\_id: Filter by jurisdiction

## Response:

### Includes:

- User information
- List of completed surveys with:
  - Site characteristics from database
  - Morphological properties
  - Survey images
  - GeoJSON geometry and properties
  - Upload timestamps

```
get(request, user_id, jurisdiction_id) \[source\]
```

```
permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]
```

---

```
class authapp.pointgeojsondata.JurisdictionAPIView(**kwargs) \[source\]
```

Bases: `APIView`

API View for managing jurisdiction data (CRUD operations).

## Permissions:

- Only Superadmin (SU) can access this endpoint

## Endpoints:

GET /jurisdictions/ - List all jurisdictions  
 GET /jurisdictions/{assigned\_district}/ - Retrieve specific jurisdiction  
 POST /jurisdictions/ - Create new jurisdiction  
 POST /jurisdictions/{assigned\_district}/ - Update existing jurisdiction  
 DELETE /jurisdictions/{assigned\_district}/ - Delete jurisdiction

## Boundary File Support:

- KML files (.kml)
- Shapefile ZIP archives (.zip containing .shp)

```
delete(request, assigned_district, format=None) \[source\]
```

```
get(request, assigned_district=None, format=None) \[source\]
```

```
permission_classes= [class 'rest\_framework.permissions.IsAuthenticated'>]
```

```
post(request, assigned_district=None, format=None) \[source\]
```

---

```
class authapp.pointgeojsondata.JurisdictionUpdateAPIView(**kwargs) \[source\]
```

Bases: `APIView`

API View for managing user-jurisdiction assignments.

## Permissions:

- SU can manage jurisdictions for any user
- AU can manage jurisdictions for UU users only

## Endpoints:

GET /user-jurisdictions/{user\_id}/ - Get user's assigned jurisdictions  
 POST /user-jurisdictions/ - Replace user's jurisdictions (delete old, create new)

## Features:

- Role-based permission checking
- Atomic transaction handling
- Email jurisdiction assignment

```
can_edit(actor, target) \[source\]
```

```
get(request, user_id, format=None) \[source\]
```

```
permission_classes= [class 'rest\_framework.permissions.IsAuthenticated'>]
```

`post(request, format=None)` [\[source\]](#)

---

`class authapp.pointgeojsondata.LabAssignedPointsAPIView(**kwargs)` [\[source\]](#)

Bases: `APIView`

Retrieve assigned lab survey points for Lab Analysts within a jurisdiction.

**Permissions:** - Only users with *designation="LAB Analyst"* - Must belong to the specified jurisdiction

**Path Parameters:** - *jurisdiction\_id*: ID of the jurisdiction to filter assignments

**Response Structure:** `` {

```
    "assigned_points": [
        {
            "sosam_code": str, "status": str, "images": list[str], // Survey images URLs ...
            serializer_fields
        }
    ]
}
```

`get(request, jurisdiction_id)` [\[source\]](#)

`permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]`

---

`class authapp.pointgeojsondata.LabAssignmentCreateAPIView(**kwargs)` [\[source\]](#)

Bases: `APIView`

Create lab survey assignments from SU/AU users to Lab Analysts.

**Permissions:** - AU: Can only assign within their accessible jurisdictions - SU: Can assign to any Lab Analyst in any jurisdiction

**Request Body (LabAssignmentCreateSerializer):** - *assigned\_to*: Profile of Lab Analyst (must belong to jurisdiction) - *jurisdiction*: Jurisdiction for the assignment - *sosam\_codes*: List of codes to assign

**Response:** `` {

```
    "message": "Assignments created successfully", "created_count": int, "assigned_codes":
    list[str]
```



```
}
```

```
permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]
```

```
post(request) \[source\]
```

---

```
class authapp.pointgeojsondata.LabSurveyDataAPIView(**kwargs) \[source\]
```

Bases: `APIView`

Lab Analysts upload XML analysis data for assigned survey points.

**Permissions:** - Only users with *designation*="LAB Analyst" can upload

**Request Body:** - *sosam\_code*: str (required) - Assigned survey point code - *xml\_file*: File (required) - XML with horizon analysis data

**Validation:** - Verifies assignment exists and user is assigned to it - Parses XML structure expecting *<sosam\_code>* root with horizon elements - Stores data atomically (DB + file system)

**Creates/Updates:** - LabSurveyData record - LabHorizonData records for each horizon in XML

**Response:** `` {

    "message": str, "stored\_at": str, "updated": bool

```
}
```

```
parser_classes  
= [<class 'rest_framework.parsers.MultiPartParser'>, <class 'rest_framework.parsers.FormParser'>]
```

```
permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]
```

```
post(request) \[source\]
```

---

```
class authapp.pointgeojsondata.LabSurveyDetailsAPIView(**kwargs) \[source\]
```

Bases: `APIView`

Retrieve detailed lab survey data for a specific user within a jurisdiction.

**Permissions:** - UU: Can only view their own lab data - AU/SU: Can view any user's lab data within jurisdiction

**Path Parameters:** - *jurisdiction\_id*: ID of the jurisdiction to filter - *user\_id*: ID of the user whose lab surveys to retrieve

Response Structure: `` {

```
    "user": {
        "username": str, "full_name": str, "role": str, "email": str, "user_id": str
    }, "surveys": [
        {
            "sosam_code": str, "status": "assigned"|"lab-in-progress"|"completed",
            "lab_horizons": list[dict], "geometry": list[float]|null, "properties": dict|null
        }
    ]
}
```

```
get(request, jurisdiction_id, user_id) [source]
```

```
permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]
```

---

**class** authapp.pointgeojsondata.LabSurveyPointsAPIView(\*\*kwargs) [source]

Bases: `APIView`

Retrieve lab survey points based on user role and jurisdiction.

**Permissions:** - AU: Only sees points reserved by themselves in the specified jurisdiction -  
SU: Sees all survey points across all jurisdictions

**Path Parameters:** - jurisdiction\_id: ID of the jurisdiction to filter (AU only)

**Returns:** Serialized list of SurveyData objects using LabSurveyPointsSerializer

```
get(request, jurisdiction_id) [source]
```

```
permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]
```

---

**class** authapp.pointgeojsondata.ManagePointAssignmentAPIView(\*\*kwargs) [source]

Bases: `APIView`

API View for managing point assignments (assign/reassign/unallocate).

**Permissions:**

- AU: Manage points within own jurisdictions
- SU: Manage all points

## Operations:

1. Unallocate: Set assigned\_to=None (when assigned\_to\_id is null)
2. Assign: Create new assignment
3. Reallocate: Update existing assignment to new UU

## Request Body:

- sosam\_codes: List of SOSAM codes
- assigned\_to: Target UU user ID (null for unallocate)

## Validation:

- AU can only manage points in their jurisdictions
- UU must belong to point's jurisdiction
- Point must exist in reservation

```
permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]
```

```
post(request) \[source\]
```

---

```
class authapp.pointgeojsondata.PointJsonAPIView(**kwargs) \[source\]
```

Bases: `APIView`

API View for retrieving POINT and POLY JSON files for assigned surveys.

## Permissions:

- Only UU users assigned to the point can access

## Path Parameters:

- sosam\_code: The survey point code

## Response:

### JSON object containing:

- point: Contents of {sosam\_code}\_SAMPLE\_POINT.json
- poly: Contents of {sosam\_code}\_SAMPLE\_POLY.json

Returns error messages for missing/invalid files

```
get(request, sosam_code) \[source\]
```

```
permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]
```

---

```
class authapp.pointgeojsondata.PointJsonZipDownloadAPIView(**kwargs) \[source\]
```

Bases: `APIView`

API View for downloading POINT and POLY JSON files as a ZIP archive.

#### Permissions:

- Only UU users assigned to the point can download

#### Path Parameters:

- `sosam_code`: The survey point code

#### Response:

##### ZIP file containing:

- `{sosam_code}_SAMPLE_POINT.json`
- `{sosam_code}_SAMPLE_POLY.json`

Returns 404 if files are missing Returns 403 if user is not assigned to the point

```
get(request, sosam_code) \[source\]
```

```
permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]
```

---

```
class authapp.pointgeojsondata.ReservePointAPIView(**kwargs) \[source\]
```

Bases: `APIView`

API View for reserving survey points (batch operation).

#### Permissions:

- Only AU users can reserve points

#### Features:

- Prefix-based jurisdiction detection (SNGP, SYVM, SSLP)
- Automatic folder structure creation (INPUT\_DATA, SURVEY\_DATA, ANALYSIS\_DATA)
- JSON file generation for point and polygon data
- Fallback to single jurisdiction if prefix not mapped

#### Request Body:

##### List of objects containing:

- `code`: SOSAM code (e.g., "SNGP123456")
- `point`: GeoJSON point data
- `polygon`: GeoJSON polygon data

```
permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]
```

```
post(request) \[source\]
```

---

`class authapp.pointgeojsondata.SurveyCompleteNamesAPIView(**kwargs)` [\[source\]](#)

Bases: `APIView`

API View for retrieving a lightweight list of completed survey codes.

#### Permissions:

- UU: View own survey codes only
- AU/SU: View any user's survey codes

#### Path Parameters:

- `user_id`: Target user's profile ID
- `jurisdiction_id`: Filter by jurisdiction

#### Response:

##### Minimal data containing:

- User information
- List of `sosam_codes` only (no details)

Useful for quick lookups and dropdown population

`get(request, user_id, jurisdiction_id)` [\[source\]](#)

`permission_classes=` [`<class 'rest_framework.permissions.IsAuthenticated'>`]

---

`class authapp.pointgeojsondata.SurveyDataUploadAPIView(**kwargs)` [\[source\]](#)

Bases: `APIView`

API View for uploading field survey data (XML, images, JSON).

#### Permissions:

- UU users can upload for assigned points
- Special handling for series codes >900000 (auto-assignment)

#### Features:

- XML parsing and database storage (SiteCharacteristics, MorphologicalProperties)
- Image upload to SURVEY\_DATA folder
- JSON file upload (required for codes >900000)
- Automatic folder structure creation
- XML node renaming for series codes
- Atomic database operations

#### Request Body (multipart/form-data):

- `sosam_code`: Survey point code
- `xml_file`: Survey data XML file

- images: Multiple image files (JPG/PNG)
- json\_file: (Required for codes >900000) Point JSON data

### Special Cases:

- Codes >900000: Auto-creates PointAssignment and PointReservation
- Requires admin mapping for auto-assignment
- Generates unique codes if conflicts exist

```
parser_classes
= [<class 'rest_framework.parsers.MultiPartParser'>, <class 'rest_framework.parsers.FormParser'>]
```

```
permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]
```

```
post(request) \[source\]
```

---

```
class authapp.pointgeojsondata.UUAssignedPointsAPIView(**kwargs) \[source\]
```

Bases: `APIView`

API View for UU users to view their assigned survey points.

### Permissions:

- Only UU users can access

### Path Parameters:

- jurisdiction\_id: Filter assignments by jurisdiction

### Response:

#### Includes:

- Point assignment details
- Site characteristics from database
- Morphological properties
- Survey images from SURVEY\_DATA folder

```
get(request, jurisdiction_id) \[source\]
```

```
permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]
```

---

```
class authapp.pointgeojsondata.UserSurveyDetailsAPIView(**kwargs) \[source\]
```

Bases: `APIView`

Retrieve detailed survey data for a specific user within a jurisdiction.

**Permissions:** - UU: Can only view their own survey data - AU/SU: Can view any user's survey data within their jurisdiction

**Path Parameters:** - jurisdiction\_id: ID of the jurisdiction to filter surveys - user\_id: ID of the user whose surveys to retrieve

```
get(request, jurisdiction_id, user_id) [source]
```

```
permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]
```

---

```
authapp.pointgeojsondata.user_in_group(user, group_name) [source]
```

Helper to check if a user belongs to a group.

## authapp.serializers module

---

```
class authapp.serializers.AUSerializer(*args, **kwargs) [source]
```

Bases: `UserListSerializer`

Serializer for Admin Users (AU) that extends UserListSerializer.

Adds a computed field `uu_users` which lists all Area Users (UU) created by this Admin User.

### Fields:

- Inherits all fields from `UserListSerializer`:

user\_id, username, first\_name, last\_name, email, role, mobile\_no, state, district, designation, ACTIVE\_STATUS, total\_count, jurisdictions, CREATION\_DATE, created\_by, blocked\_at, unblocked\_at.

- `uu_users`: List of UU users created by this AU (nested `UUSerializer`).

### Computed Methods:

- `get_uu_users()`: Returns serialized data of all UU users created by this AU.

### Notes

- Used primarily when an SU wants to see AU users along with their UUs.
- Relies on the `UUSerializer` for nested UU data representation.

```
class Meta [source]
```

Bases: `Meta`

### fields

```
= ['user_id', 'username', 'first_name', 'last_name', 'email', 'role', 'mobile_no', 'state', 'district', 'designation', 'ACTIVE_STATUS', 'total_count', 'jurisdictions', 'CREATION_DATE', 'created_by', 'blocked_at', 'unblocked_at', 'uu_users']
```

`get_uu_users(obj)` [\[source\]](#)

---

`class authapp.serializers.AssignedLabSurveySerializer(*args, **kwargs)` [\[source\]](#)

Bases: `ModelSerializer`

Serializer for LabAssignment records assigned to Lab Analysts, including status, media information, and assignment metadata.

#### Fields:

`id` (int): LabAssignment ID. `sosam_code` (str): Unique code for the survey point. `status` (str): Status of the lab survey. “completed” if LabSurveyData exists, else “pending”. `media_info` (dict): JSON data from the INPUT\_DATA folder associated with the survey point. `assigned_by_id` (int): ID of the user who created the assignment. `assigned_by_username` (str): Username of the user who created the assignment. `assigned_to_id` (int): ID of the Lab Analyst assigned to the survey. `assigned_to_username` (str): Username of the Lab Analyst assigned to the survey. `assigned_at` (datetime): Timestamp when the assignment was created. `horizon_no` (int): Horizon number from the associated SurveyData, if available.

#### Read-only fields:

`id`, `assigned_by_id`, `assigned_by_username`, `assigned_to_id`, `assigned_to_username`

`get_status(obj)` [\[source\]](#)

Determines the status of the lab assignment. Returns “completed” if LabSurveyData exists, otherwise “pending”.

`get_media_info(obj)` [\[source\]](#)

Reads and returns JSON content from the INPUT\_DATA folder for the survey point. Returns empty dict if folder or file is missing or if any error occurs.

`class Meta` [\[source\]](#)

Bases: `object`

#### fields

`= ['id', 'sosam_code', 'status', 'media_info', 'assigned_by_id', 'assigned_by_username', 'assigned_to_id', 'assigned_to_username', 'assigned_at', 'horizon_no']`

#### model

alias of `LabAssignment`

#### read\_only\_fields

`= ['id', 'assigned_by_id', 'assigned_by_username', 'assigned_to_id', 'assigned_to_username']`



`get_media_info(obj)` [\[source\]](#)

`get_status(obj)` [\[source\]](#)

---

`class authapp.serializers.ChangePasswordSerializer(*args, **kwargs)` [\[source\]](#)

Bases: `Serializer`

Serializer for changing a user's password.

**Fields:**

- `old_password`: The current password of the user (required, write-only).
- `new_password`: The new password to set (required, write-only).
- `confirm_password`: Confirmation of the new password (required, write-only).

**Validation:**

- `validate_new_password`: Ensures the new password meets Django's password validation rules.
- `validate`: Ensures that `new_password` and `confirm_password` match.

`validate(data)` [\[source\]](#)

`validate_new_password(value)` [\[source\]](#)

---

`class authapp.serializers.ForgotPasswordRequestSerializer(*args, **kwargs)` [\[source\]](#)

Bases: `Serializer`

Serializer for requesting a password reset via email.

**Fields:**

- `email`: The email address of the user requesting a password reset.

**Usage:**

- Validates that the provided email is in a proper email format.
- Typically used in the "Forgot Password" API to initiate the OTP or reset link process.

---

`class authapp.serializers.JurisdictionSerializer(*args, **kwargs)` [\[source\]](#)

Bases: `ModelSerializer`

Serializer for the Jurisdiction model.

**Fields:**

- `id`: Primary key of the jurisdiction.

- `assigned_district`: Name of the district (must be unique).
- `boundary`: Geospatial boundary of the jurisdiction.

#### Validation:

- Ensures that the `assigned_district` is unique in the database.
- Additional validation logic can be added in the `validate` method.

```
class Meta \[source\]
```

Bases: `object`

```
fields= ['id', 'assigned_district', 'boundary']
```

```
model
```

alias of `Jurisdiction`

```
validate(data) \[source\]
```

```
class authapp.serializers.LabAssignmentCreateSerializer(*args, **kwargs) \[source\]
```

Bases: `Serializer`

Serializer for creating LabAssignment records, assigning SurveyData points to Lab Analysts within a specific jurisdiction.

#### Fields:

`sosam_codes` (List[str]): List of SurveyData `sosam_code`'s to assign. Cannot be empty.  
`assigned_to` (int): User ID of the Lab Analyst to whom the surveys will be assigned.  
`jurisdiction` (int): ID of the Jurisdiction associated with the assignment.

#### Validation:

- Only users with role "SU" or "AU" can create lab assignments.
- `assigned_to` must be a valid active Profile with designation "LAB Analyst".
- `jurisdiction` must exist.
- Each `sosam_code` must exist in SurveyData and not be already assigned.

```
validate(attrs) \[source\]
```

Ensures that the request user is authorized to assign surveys.

```
validate_assigned_to(value) \[source\]
```

Checks that the target Lab Analyst exists and is active.

```
validate_jurisdiction(value) \[source\]
```

Ensures the specified jurisdiction exists.

**validate\_sosam\_codes**(codes) [\[source\]](#)

Checks existence of SurveyData points and prevents duplicate assignments.

**create**(validated\_data) [\[source\]](#)

Creates LabAssignment records for each *sosam\_code* in an atomic transaction, updates *is\_sent\_to\_lab* and *lab\_sent\_at* on SurveyData, and returns the list of created LabAssignment instances.

**create**(validated\_data) [\[source\]](#)

**validate**(attrs) [\[source\]](#)

**validate\_assigned\_to**(value) [\[source\]](#)

**validate\_jurisdiction**(value) [\[source\]](#)

**validate\_sosam\_codes**(codes) [\[source\]](#)

---

**class** authapp.serializers.LabHorizonDataSerializer(\*args, \*\*kwargs) [\[source\]](#)

Bases: `ModelSerializer`

Serializer for LabHorizonData model representing soil horizon properties.

This serializer handles numeric and chemical properties of a soil horizon associated with a lab survey point.

**Fields:**

id (int): Primary key of the horizon record (read-only). horizon\_code (str): Unique code for the soil horizon. sand (float): Percentage of sand content. silt (float): Percentage of silt content. clay (float): Percentage of clay content. porosity (float): Soil porosity value. ph (float): Soil pH value. ogc (float): Organic carbon content. exAl (float): Exchangeable aluminum. exH (float): Exchangeable hydrogen. exCatNa (float): Exchangeable sodium. exCatK (float): Exchangeable potassium. exCatCa (float): Exchangeable calcium. exCatMg (float): Exchangeable magnesium. cec (float): Cation exchange capacity. ec (float): Electrical conductivity. esp (float): Exchangeable sodium percentage. caco3 (float): Calcium carbonate content. availN (float): Available nitrogen. availP (float): Available phosphorus. availK (float): Available potassium. availFe (float): Available iron. availMn (float): Available manganese. availZn (float): Available zinc. availCu (float): Available copper. availB (float): Available boron.

**class** Meta [\[source\]](#)

Bases: `object`

```
fields
= ['id', 'horizon_code', 'sand', 'silt', 'clay', 'porosity', 'ph', 'ogc', 'exAl', 'exH', 'exCatNa', 'exCatK',
'exCatCa', 'exCatMg', 'cec', 'ec', 'esp', 'caco3', 'availN', 'availP', 'availK', 'availFe', 'availMn', 'availZn',
'availCu', 'availB']
```

**model**

alias of `LabHorizonData`

```
read_only_fields= ('id',)
```

---

```
class authapp.serializers.LabSurveyPointsSerializer(*args, **kwargs) \[source\]
```

Bases: `ModelSerializer`

Serializer for SurveyData points sent to the lab.

This serializer provides detailed information about each survey point, including uploader info, assignment status, media data, and lab assignment details.

#### Fields:

`id` (int): Primary key of the SurveyData record. `sosam_code` (str): Unique identifier for the survey point. `uploader` (User): User who uploaded the survey data (read-only). `uploaded_by_username` (str): Username of the uploader (read-only). `uploaded_at` (datetime): Timestamp of upload (read-only). `is_sent_to_lab` (bool): Flag indicating if the point has been sent to the lab (read-only). `lab_sent_at` (datetime): Timestamp when sent to lab (read-only). `status` (str): Computed status of the survey point; "completed", "assigned", or "pending". `lab_assignment_info` (dict | None): Details of lab assignment if exists, including assigned user and assignment timestamp. `media_info` (dict): JSON data extracted from the point's INPUT\_DATA folder, if available.

```
class Meta \[source\]
```

Bases: `object`

```
fields
= ['id', 'sosam_code', 'uploader', 'uploaded_by_username', 'uploaded_at', 'is_sent_to_lab',
'lab_sent_at', 'status', 'lab_assignment_info', 'media_info']
```

**model**

alias of `SurveyData`

```
read_only_fields
= ['uploader', 'uploaded_by_username', 'uploaded_at', 'is_sent_to_lab', 'lab_sent_at', 'status']
```

```
get_lab_assignment_info(obj) \[source\]
```

Get lab assignment details if exists

`get_media_info(obj)` [\[source\]](#)

`get_status(obj)` [\[source\]](#)

---

`class authapp.serializers.LoginSerializer(*args, **kwargs)` [\[source\]](#)

Bases: `Serializer`

Serializer for user login.

#### Fields:

- username: The username of the user.
- password: The user's password (write-only).

#### Validation:

- Authenticates the user using Django's *authenticate*.
- Ensures that the user's profile *ACTIVE\_STATUS* is "Active".
- Raises *ValidationError* if credentials are invalid or account is not active.

#### Returns:

- The validated user instance in *attrs["user"]* for use in login views.

`validate(attrs)` [\[source\]](#)

---

`class authapp.serializers.MorphologicalPropertiesSerializer(*args, **kwargs)` [\[source\]](#)

Bases: `ModelSerializer`

Serializer for the *MorphologicalProperties* model.

Excludes the 'id' and 'survey\_data' fields. Used for creating, updating, or viewing morphological property details associated with a survey data record.

`class Meta` [\[source\]](#)

Bases: `object`

`exclude= ['id', 'survey_data']`

`model`

alias of `MorphologicalProperties`

---

`class authapp.serializers.PasswordResetConfirmSerializer(*args, **kwargs)` [\[source\]](#)

Bases: `Serializer`

Serializer for confirming a password reset and updating user profile information.

#### Fields:

- `new_password1`: The new password to set (write-only, required).
- `new_password2`: Confirmation of the new password (write-only, required).
- `first_name`: Optional first name for the user (can be blank).
- `last_name`: Optional last name for the user (can be blank).
- `mobile_no`: Required mobile number (max 10 characters).

#### Validation:

- Ensures that `new_password1` and `new_password2` match.
- Validates the new password using Django's `validate_password` function.

**`validate(attrs)`** [\[source\]](#)

---

```
class authapp.serializers.PointAssignmentSerializer(*args, **kwargs) \[source\]
```

Bases: `ModelSerializer`

Serializer for the `PointAssignment` model, representing assignments of points (`sosam_code`) to users with optional media and survey tracking.

#### Fields:

- `id`: Primary key of the assignment.
- `sosam_code`: Unique code of the point being assigned.
- `assigned_by_id`: ID of the user who assigned the point.
- `assigned_by_username`: Username of the assigner.
- `assigned_to_id`: ID of the user to whom the point is assigned.
- `assigned_to_username`: Username of the assignee.
- `assigned_at`: Timestamp when the assignment was made.
- `is_uploaded`: Boolean indicating if survey data exists for this point.
- `media_info`: JSON content from the `INPUT_DATA` folder of the point.

#### - `get_is_uploaded`

Checks database for survey data existence.

#### - `get_media_info`

Reads JSON from `INPUT_DATA` folder for the point, supports multiple jurisdictions.

#### - `validate`

Ensures `sosam_code` is provided and prevents duplicate initial assignments.

## - create

Creates a new PointAssignment using context for assigned\_by, assigned\_to, and jurisdiction\_id.

## Notes

- Media folder path supports older points without jurisdiction folders.
- Read-only fields prevent modification of assignment metadata.
- **Intended to be used with context keys:**

```
{“assigned_by”: User instance, “assigned_to”: User instance, “jurisdiction_id”: int}
```

```
class Meta \[source\]
```

Bases: `object`

### fields

```
= ['id', 'sosam_code', 'assigned_by_id', 'assigned_by_username', 'assigned_to_id',  
  'assigned_to_username', 'assigned_at', 'is_uploaded', 'media_info']
```

### model

alias of `PointAssignment`

### read\_only\_fields

```
= ['id', 'assigned_by_id', 'assigned_by_username', 'assigned_to_id', 'assigned_to_username',  
  'assigned_at']
```

```
create(validated_data) \[source\]
```

We have a bit of extra checking around this in order to provide descriptive messages when something goes wrong, but this method is essentially just:

```
return ExampleModel.objects.create(**validated_data)
```

If there are many to many fields present on the instance then they cannot be set until the model is instantiated, in which case the implementation is like so:

```
example_relationship = validated_data.pop('example_relationship') instance =  
ExampleModel.objects.create(**validated_data) instance.example_relationship =  
example_relationship return instance
```

The default implementation also does not handle nested relationships. If you want to support writable nested relationships you'll need to write an explicit `.create()` method.

```
get_is_uploaded(obj) \[source\]
```

`get_media_info(obj)` [\[source\]](#)

`validate(attrs)` [\[source\]](#)

---

`class authapp.serializers.PointReservationSerializer(*args, **kwargs)` [\[source\]](#)

Bases: `ModelSerializer`

Serializer for PointReservation model, providing detailed information about a reserved point, including assignment, status, media, and jurisdiction.

#### Fields:

- `id`: Primary key of the PointReservation.
- `sosam_code`: Unique code of the reserved point.
- `reserved_by`: User who reserved the point (ID).
- `reserved_by_username`: Username of the reserver.
- `reserved_at`: Timestamp when the point was reserved.
- `jurisdiction_name`: Name of the jurisdiction the point belongs to.
- `media_info`: JSON content from the point's INPUT\_DATA folder.
- `assigned_to`: User ID to whom the point is assigned.
- `assigned_to_username`: Username of the assignee.
- **status**: Current status of the point. Possible values:
  - "unassigned" – not yet assigned
  - "assigned" – assigned but survey not completed
  - "completed" – survey data exists

#### - `get_media_info`

Reads JSON from INPUT\_DATA folder for the point.

#### - `get_assigned_to` / `get_assigned_to_username`

Fetches assignment info from context.

#### - `get_status`

Determines the current status of the point.

#### Notes

- Supports multiple jurisdictions.
- Handles older points without jurisdiction folders gracefully.
- Assignments can be passed in the serializer context as: `{"assignments": { "sosam_code": AssignmentInstance }}`

`class Meta` [\[source\]](#)

Bases: `object`



```
fields
= ['id', 'sosam_code', 'reserved_by', 'reserved_by_username', 'reserved_at', 'jurisdiction_name',
  'media_info', 'assigned_to', 'assigned_to_username', 'status']
```

**model**

alias of `PointReservation`

```
read_only_fields
= ['reserved_by', 'reserved_by_username', 'reserved_at', 'jurisdiction_name', 'assigned_to',
  'assigned_to_username', 'status']
```

`get_assigned_to(obj)` [\[source\]](#)

`get_assigned_to_username(obj)` [\[source\]](#)

`get_media_info(obj)` [\[source\]](#)

`get_status(obj)` [\[source\]](#)

---

```
class authapp.serializers.ProfileTransferSerializer(*args, **kwargs) \[source\]
```

Bases: `ModelSerializer`

Serializer for transferring a UU user's profile to another user.

**Fields:**

- `assigned_to`: The user to whom the profile is being reassigned. This field is required.

**Usage:**

- Used in API endpoints to update the *assigned\_to* field of a Profile instance.

```
class Meta \[source\]
```

Bases: `object`

```
extra_kwargs= {'assigned_to': {'required': True}}
```

```
fields= ['assigned_to']
```

**model**

alias of `Profile`

---

`class authapp.serializers.ResetPasswordSerializer(*args, **kwargs)` [\[source\]](#)

Bases: `Serializer`

Serializer for resetting a user's password.

#### Fields:

- email: The email address of the user whose password is being reset.
- new\_password: The new password the user wants to set.
- confirm\_password: A confirmation of the new password to ensure it matches.

#### Validation:

- Ensures that *new\_password* and *confirm\_password* are identical.

#### Usage:

- Typically used after OTP verification or password reset token validation.

`validate(data)` [\[source\]](#)

---

`class authapp.serializers.SignupSerializer(*args, **kwargs)` [\[source\]](#)

Bases: `Serializer`

Serializer for creating new users (Admin User or Area User) via invitation.

#### Fields:

- email: Email address of the user (required, must be unique).
- role: Role of the user; choices are "AU" (Admin User) or "UU" (Area User).
- designation: Optional designation for the user; required for UU users.
- mobile\_no: Optional mobile number (minimum 10 digits).
- jurisdiction: List of Jurisdiction IDs assigned to the user. For UU users, can inherit from the creator AU if not provided.

#### Validation:

- validate\_email: Ensures email is unique and not associated with a frozen user.
- validate\_mobile\_no: Ensures mobile number is at least 10 digits if provided.
- **validate: Enforces role-based permissions:**
  - Only SU can create AU users.
  - Only AU can create UU users.
  - UU users must have a designation.
  - UU users inherit jurisdiction from AU if not explicitly provided.

`ROLE_CHOICES= [('AU', 'Admin User'), ('UU', 'Area User')]`

`validate(data)` [\[source\]](#)

`validate_email(value)` [\[source\]](#)

`validate_mobile_no(value)` [\[source\]](#)

Optional: Add validation for mobile number format

---

`class authapp.serializers.SiteCharacteristicsSerializer(*args, **kwargs)` [\[source\]](#)

Bases: `ModelSerializer`

Serializer for the SiteCharacteristics model.

Excludes the 'id' and 'survey\_data' fields. Used for creating, updating, or viewing site characteristic details associated with a survey data record.

`class Meta` [\[source\]](#)

Bases: `object`

`exclude= ['id', 'survey_data']`

`model`

alias of `SiteCharacteristics`

---

`class authapp.serializers.SurveyDataUploadSerializer(*args, **kwargs)` [\[source\]](#)

Bases: `Serializer`

Serializer for uploading survey data for a given point (sosam\_code).

#### Fields:

- `sosam_code` (str): Required. Unique identifier of the point.
- `xml_file` (File): Required. XML file containing survey data.
- `json_file` (File): Optional. JSON file containing additional survey data.
- `images` (List[Image]): Optional. List of image files associated with the point.

#### Validation:

- `images`: At least one image is required if provided, maximum of 5 images allowed.

#### Notes

- This serializer does not create a model instance; it is intended for file upload handling and validation.
- The validated data can be used in views to save files, parse XML/JSON, and create related SurveyData or PointAssignment records.

`validate_images(images)` [\[source\]](#)

---

`class authapp.serializers.SurveyDataViewSerializer(*args, **kwargs)` [\[source\]](#)

Bases: `Serializer`

Serializer for viewing uploaded survey data for a specific point (sosam\_code).

#### Fields:

- `sosam_code` (str): Unique identifier of the point.
- `xml_file` (str | None): Path or name of the uploaded XML file. Can be null.
- `image_files` (List[str]): List of uploaded image file names or paths. Can be empty.
- `uploaded_by` (str | None): Username of the user who uploaded the survey data. Can be null.
- `uploaded_at` (datetime | None): Timestamp when the survey data was uploaded. Can be null.

#### Notes

- This serializer is primarily read-only and used for presenting survey data details in API responses.
- All fields are optional in the sense that they can be null or empty.

---

`class authapp.serializers.UUSerializer(*args, **kwargs)` [\[source\]](#)

Bases: `UserListSerializer`

---

`class authapp.serializers.UserListSerializer(*args, **kwargs)` [\[source\]](#)

Bases: `ModelSerializer`

Serializer for listing users with their profile and jurisdiction details.

This serializer is based on the Profile model, but also includes related User fields and computed properties such as role, total assigned points, and multiple jurisdictions.

#### Fields:

- `user_id`: ID of the associated User instance.
- `username`, `first_name`, `last_name`, `email`: From related User model.
- `role`: Name of the first group the user belongs to (computed).
- `mobile_no`, `state`, `district`, `designation`, `ACTIVE_STATUS`: From Profile.
- `total_count`: Number of PointAssignment objects assigned to this user (computed).
- `jurisdictions`: List of assigned jurisdictions with their IDs and district names (computed).
- `CREATION_DATE`: Profile creation timestamp, formatted as "dd-mm-YYYY HH:MM:SS".
- `created_by`: Username of the creator of this profile.
- `blocked_at`, `unblocked_at`: Timestamps for block/unblock actions.

#### Computed Methods:

- `get_role()`: Returns the name of the first group the user belongs to.
- `get_user_id()`: Returns the ID of the associated User.
- `get_total_count()`: Returns the count of PointAssignment objects assigned to the user.
- `get_jurisdictions()`: Returns a list of dictionaries containing assigned jurisdiction IDs and names.

## Notes

- Intended for listing multiple users with profile and jurisdiction metadata.
- Read-only fields include `ACTIVE_STATUS`, `CREATION_DATE`, `blocked_at`, `unblocked_at`, and computed fields.

`class Meta` [\[source\]](#)

Bases: `object`

### fields

= ['user\_id', 'username', 'first\_name', 'last\_name', 'email', 'role', 'mobile\_no', 'state', 'district', 'designation', 'ACTIVE\_STATUS', 'total\_count', 'jurisdictions', 'CREATION\_DATE', 'created\_by', 'blocked\_at', 'unblocked\_at']

### model

alias of `Profile`

`get_jurisdictions(obj)` [\[source\]](#)

`get_role(obj)` [\[source\]](#)

`get_total_count(obj)` [\[source\]](#)

`get_user_id(obj)` [\[source\]](#)

---

`class authapp.serializers.UserProfileSerializer(*args, **kwargs)` [\[source\]](#)

Bases: `ModelSerializer`

Serializer for representing detailed user profile information.

Includes fields from both the User model and the related Profile model, along with computed fields like `role`, `created_by`, and associated jurisdictions.

## Fields:

- `username`, `email`, `first_name`, `last_name`: From User model.
- `user_id`: ID of the User instance.
- `role`: Name of the first group the user belongs to (e.g., SU, AU, UU).

- `created_by`: Username of the user who created this profile.
- `mobile_no`, `state`, `district`, `department`, `designation`: From related Profile.
- `ACTIVE_STATUS`: Current status of the user (Active, Inactive, Frozen, etc.).
- `blocked_at`, `unblocked_at`: Timestamps for blocking/unblocking actions.
- `deleted_users_count`: Number of users deleted by this user.
- `CREATION_DATE`: Profile creation timestamp (formatted as “dd-mm-YYYY HH:MM:SS”).
- `jurisdiction`: List of jurisdictions assigned to the user, including boundary details.

### Computed Fields:

- `get_role()`: Returns the name of the user’s first group.
- `get_user_id()`: Returns the user’s ID.
- `get_created_by()`: Returns the username of the creator.
- `get_jurisdiction()`: Returns a list of dictionaries containing jurisdiction details and GeoJSON-like boundary coordinates.

### Read-only:

All fields except those directly editable on the User model (e.g., `first_name`, `last_name`).

```
class Meta \[source\]
```

Bases: `object`

#### fields

```
= ['username', 'email', 'first_name', 'last_name', 'user_id', 'created_by', 'role', 'mobile_no', 'state',
'district', 'department', 'designation', 'ACTIVE_STATUS', 'jurisdiction', 'CREATION_DATE',
'blocked_at', 'unblocked_at', 'deleted_users_count']
```

#### model

alias of `User`

#### read\_only\_fields

```
= ['username', 'email', 'user_id', 'created_by', 'role', 'ACTIVE_STATUS', 'CREATION_DATE',
'blocked_at', 'unblocked_at', 'deleted_users_count']
```

```
get_created_by(obj) \[source\]
```

```
get_jurisdiction(obj) \[source\]
```

```
get_role(obj) \[source\]
```

```
get_user_id(obj) \[source\]
```

---

**class** `authapp.serializers.UserSurveyPointSerializer(*args, **kwargs)` [\[source\]](#)

Bases: `Serializer`

Serializer for a user's survey point data.

#### Fields:

`point_name` (str): Name or identifier of the survey point. `status` (str): Current status of the point (e.g., "completed", "assigned", "unassigned"). `xml_file` (str | None): Path or URL to the associated XML file, if any. `images` (list[str]): List of image file paths or URLs associated with the point. `input_data` (list[dict]): Optional JSON input data related to the point, allowing

multiple structured entries per point.

---

**class** `authapp.serializers.VerifyOTPSerializer(*args, **kwargs)` [\[source\]](#)

Bases: `Serializer`

Serializer for verifying a One-Time Password (OTP) during password reset.

#### Fields:

- `email`: The email address of the user attempting to verify the OTP.
- `otp`: The 6-digit OTP sent to the user's email for verification.

#### Usage:

- Validates the email format.
- Ensures the OTP is provided and does not exceed 6 characters.
- Typically used in the "Verify OTP" step of the password reset workflow.

## authapp.tests module

## authapp.urls module

## authapp.utils module

---

**authapp.utils.send\_brevo\_email**(*to\_email*: str, *subject*: str, *html\_content*: str) [\[source\]](#)

Send a transactional email using Brevo (formerly SendinBlue) API.

**Parameters:**

- `to_email` (str) – Recipient email address.
- `subject` (str) – Subject line of the email.
- `html_content` (str) – HTML content of the email body.

**Returns:** True if email was successfully sent, False if an error occurred.

**Return type:** bool

#### Notes

- Requires the environment variables:

- BREVO\_API\_KEY: API key for Brevo.
- DEFAULT\_FROM\_EMAIL: Sender's email address.
- The sender name is hardcoded as "SOIL MAPPING PROJECT".
- Exceptions from the Brevo API are caught and logged.

## authapp.views module

---

**class** `authapp.views.ChangePasswordView(**kwargs)` [\[source\]](#)

Bases: `UpdateAPIView`

API endpoint that allows authenticated users to change their password.

### Permissions:

- Only authenticated users can access this endpoint.

### Behavior:

1. Validates the provided old password.
2. Updates the user's password to the new password.
3. Keeps the current session authenticated after password change using `update_session_auth_hash`.

### Notes

- Ensures that a user cannot change their password without providing the correct current password.
- Session remains valid after password change to prevent the user from being logged out.

**get\_object**(`queryset=None`) [\[source\]](#)

Returns the object the view is displaying.

You may want to override this if you need to provide non-standard queryset lookups. Eg if objects are referenced using multiple keyword arguments in the url conf.

### model

alias of `User`

**permission\_classes**= [`<class 'rest_framework.permissions.IsAuthenticated'>`]

### serializer\_class

alias of `ChangePasswordSerializer`

**update**(`request, *args, **kwargs`) [\[source\]](#)



---

`class authapp.views.DeleteUserView(**kwargs)` [\[source\]](#)

Bases: `APIView`

API endpoint to delete a user and their associated profile.

Permission rules: - SU (Superuser):

- Can delete any user (including AU or UU)
- **AU (Admin User):**
  - Cannot delete themselves
  - Can delete only UU users they created
- **Others:**
  - No deletion permissions

`delete(request, user_id)` [\[source\]](#)

`permission_classes=` [`<class 'rest_framework.permissions.IsAuthenticated'>`]

---

`class authapp.views.DownloadAPK(**kwargs)` [\[source\]](#)

Bases: `APIView`

Serves the latest APK file stored in the MEDIA\_ROOT/APK directory.

It searches the folder for any .apk file and provides it for download as an attachment.

Raises 404 if no APK file is found.

`get(request)` [\[source\]](#)

`permission_classes=` [`<class 'rest_framework.permissions.AllowAny'>`]

---

`class authapp.views.ExpiringPasswordResetAPIView(**kwargs)` [\[source\]](#)

Bases: `APIView`

API view to validate and process expiring password reset / account activation links.

Supports: - GET → Validate token and return user metadata - POST → Set password, activate account, assign roles & groups

`dispatch(request, *args, **kwargs)`

`.dispatch()` is pretty much the same as Django's regular dispatch, but with extra hooks for startup, finalize, and exception handling.

`get(request, uidb64, token)` [\[source\]](#)

Validate the password reset / activation link.

Steps: - Decode UID (Profile ID) - Check profile existence - Verify expiry based on profile creation date - Validate token

Returns: - User metadata if valid - Error message if invalid or expired

#### **parser\_classes**

```
= [<class 'rest_framework.parsers.JSONParser'>, <class 'rest_framework.parsers.FormParser'>, <class 'rest_framework.parsers.MultiPartParser'>]
```

#### **post(request, uidb64, token)** [\[source\]](#)

Finalize account setup: - Validate token - Set password - Activate user & profile - Assign role-based permissions and groups - Send welcome email

---

```
class authapp.views.ForgotPasswordSendOTP(**kwargs) \[source\]
```

Bases: `APIView`

API endpoint to request a One-Time Password (OTP) for password reset.

#### **Permissions:**

- Accessible by anyone (AllowAny).

#### **Behavior:**

1. Validates that the email corresponds to an active user.
2. Enforces rate limiting: maximum 1 request per minute per email.
3. Enforces a 24-hour lock: user can request password reset only once per 24 hours.
4. Generates a 6-digit OTP and stores it in cache with a 5-minute expiry.
5. Sends the OTP to the user's email via Brevo.
6. Clears any previous OTP data (verification status, used flag) before sending a new OTP.

#### **Notes**

- OTP is valid for 5 minutes.
- Rate-limiting and 24-hour lock are enforced via cache.
- This endpoint does not reveal if an email exists in the system beyond the error messages.

```
permission_classes= [<class 'rest_framework.permissions.AllowAny'>]
```

#### **post(request)** [\[source\]](#)

---

```
class authapp.views.FreezeUIView(**kwargs) \[source\]
```

Bases: `APIView`

API endpoint to freeze (temporarily disable) a user account.

Permission rules: - SU (Superuser):

- Can freeze any user
- **AU (Admin User):**
  - Cannot freeze themselves
  - Can freeze only UU users they created
- **UU or others:**
  - No permission to freeze users

```
permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]
```

```
post(request, user_id) \[source\]
```

---

```
class authapp.views.FrozenUserListView(**kwargs) \[source\]
```

Bases: `ListAPIView`

API endpoint to list frozen users based on requester role.

Visibility rules: - SU (Superuser):

- Can view all frozen AU and UU users
- **AU (Admin User):**
  - Can view only frozen UU users they created
- **Others:**
  - No access

```
get_queryset() \[source\]
```

Get the list of items for this view. This must be an iterable, and may be a queryset. Defaults to using *self.queryset*.

This method should always be used rather than accessing *self.queryset* directly, as *self.queryset* gets evaluated only once, and those results are cached for all subsequent requests.

You may want to override this if you need to provide different querysets depending on the incoming request.

(Eg. return a list of items that is specific to the user)

```
permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]
```

**serializer\_class**

alias of `UserListSerializer`

---

```
class authapp.views.LoginAPIView(**kwargs) \[source\]
```

Bases: `APIView`

API endpoint for user authentication.

This endpoint: - Authenticates a user using provided credentials - Generates JWT access and refresh tokens - Returns user identity, role, and profile details

Authentication & Permissions: - authentication\_classes: Disabled (public endpoint) - permission\_classes: Disabled (anyone can log in)

```
authentication_classes= []
```

```
dispatch(request, *args, **kwargs)
```

`.dispatch()` is pretty much the same as Django's regular dispatch, but with extra hooks for startup, finalize, and exception handling.

```
permission_classes= []
```

```
post(request) \[source\]
```

---

```
class authapp.views.LogoutAPIView(**kwargs) \[source\]
```

Bases: `APIView`

API endpoint to log out a user by blacklisting their refresh token.

**Permission:**

- Requires authentication (user must be logged in)

**Notes**

- The client must provide a valid refresh token in the request body.
- Blacklisted tokens cannot be used to generate new access tokens.

```
permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]
```

```
post(request) \[source\]
```

---

```
class authapp.views.Particular_UserListView(**kwargs) \[source\]
```

Bases: `ListAPIView`

List users based on the role of the requesting user.

#### Permissions:

- SU: Can see all AU users (and optionally their UUs if using nested serializer).
- AU: Can see only UU users they created.
- UU: No access.

`get_queryset()` [\[source\]](#)

Return queryset based on user role:

- SU: all AU users
- AU: their own UU users
- Others: raise `PermissionDenied`

`get_serializer_class()` [\[source\]](#)

Return serializer based on user role:

- SU: `AUSerializer` (to include AU-specific info)
- AU: `UserListSerializer` (simple list of UUs)

`permission_classes=` [`<class 'rest_framework.permissions.IsAuthenticated'>`]

---

`class authapp.views.ProfileTransferView(**kwargs)` [\[source\]](#)

Bases: `APIView`

Transfer a UU profile from one AU/SU to another.

Permissions: - User must be authenticated - Only SU users are allowed to perform transfers

Validation: - Target UU must exist - Requesting user must share at least one jurisdiction with the UU

`permission_classes=` [`<class 'rest_framework.permissions.IsAuthenticated'>`]

`post(request)` [\[source\]](#)

---

`class authapp.views.ResendInviteView(**kwargs)` [\[source\]](#)

Bases: `APIView`

API endpoint to resend an account activation / password setup invite.

Rules: - Only inactive users can be re-invited - Only the creator of the user can resend the invite - Resets invite expiry by updating profile creation timestamp

```
permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]
```

```
post(request, *args, **kwargs) \[source\]
```

---

```
class authapp.views.ResetPasswordView(**kwargs) \[source\]
```

Bases: `APIView`

Handles resetting a user's password after OTP verification.

This view ensures that the OTP has been verified, the user is active, and the OTP has not been used before. Once the password is successfully reset, it marks the OTP as used and enforces a 24-hour lock on further password reset requests.

```
permission_classes= [<class 'rest_framework.permissions.AllowAny'>]
```

```
post(request) \[source\]
```

---

```
class authapp.views.SignupView(**kwargs) \[source\]
```

Bases: `CreateAPIView`

Invite-based user signup API.

This endpoint allows authenticated users (SU/AU) to: - Create a new inactive user - Assign role-based permissions and groups - Assign jurisdictions - Send an email invite with a password setup link

```
create(request, *args, **kwargs) \[source\]
```

```
perform_create(user, role, request_user, designation, mobile_no, jurisdictions) \[source\]
```

Create profile, assign group, create jurisdictions, send invite email.

```
permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]
```

```
queryset
```

```
= <QuerySet [<User: riteshborikar1010>, <User: sahilsharma2201>, <User: pawardivya730>, <User: pradumm.microdev01>, <User: ankush.acetech1>, <User: sahil.acetech>, <User: micronetshivam>, <User: pradumm.microdev06>, <User: vedant.ace>, <User: ritesh.acetech>]>
```

```
serializer_class
```

alias of `SignupSerializer`

---

```
class authapp.views.TokenRefreshAPIView(**kwargs) \[source\]
```

Bases: `TokenRefreshView`

Custom JWT token refresh API view.

Differences from default SimpleJWT behavior: - Accepts *refresh\_token* instead of *refresh* in request body - Returns *access\_token* instead of *access* - Supports refresh token rotation transparently

```
parser_classes= [<class 'rest_framework.parsers.JSONParser'>]
```

```
post(request: Request, *args, **kwargs) [source]
```

POST /api/token/refresh/ Body: { "refresh\_token": "<refresh\_token>" } Returns: { access\_token: "...", refresh\_token: "..." }

---

```
class authapp.views.UnfreezeUserView(**kwargs) [source]
```

Bases: `APIView`

API endpoint to unfreeze (reactivate) a previously frozen user account.

Permission rules: - SU (Superuser):

- Can unfreeze any user
- **AU (Admin User):**
  - Cannot unfreeze themselves
  - Can unfreeze only UU users they created
- **UU or others:**
  - No permission to unfreeze users

```
permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]
```

```
post(request, user_id) [source]
```

---

```
class authapp.views.UpdateUserProfileView(**kwargs) [source]
```

Bases: `APIView`

API endpoint to update a user's profile and optional jurisdiction assignments.

Permissions:

- SU: Can update any user, including jurisdictions.
- AU: Can update themselves or UU users they created (excluding jurisdictions).
- UU: Can update only their own profile (excluding jurisdictions).

```
permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]
```

```
post(request, user_id) [source]
```

---

**class** `authapp.views.UpdateUserStatusAPIView(**kwargs)` [\[source\]](#)

Bases: `APIView`

API endpoint to update the ACTIVE\_STATUS of a user profile.

Roles and Permissions: - SU (Superuser): Can update status of any AU or UU user. - AU (Admin User): Can update status only for UU users they created. Cannot modify Frozen users. - UU (Regular User): Cannot update anyone's status.

Supported Status Values: - "Active": User account is active. - "Inactive": User account is inactive. - "Blocked": User account is blocked. - "Frozen": User account is frozen.

Cascading Effects: - When an AU is blocked or reactivated, all their UU users are automatically blocked or reactivated. - Emails are sent to the affected users to notify about status changes.

```
permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]
```

```
post(request) \[source\]
```

---

**class** `authapp.views.UploadShapefileToGeoServer(**kwargs)` [\[source\]](#)

Bases: `APIView`

Handles uploading one or more shapefiles to a GeoServer workspace.

Steps: 1. Validates the uploaded ZIP file containing shapefiles. 2. Extracts individual shapefiles and their related files. 3. Ensures the specified GeoServer workspace exists; creates it if not. 4. Uploads each shapefile as a separate datastore. 5. Returns a summary of successfully published shapefiles and any errors.

Supports multiple shapefiles in a single ZIP. Generates WMS preview URLs for published layers.

```
post(request, *args, **kwargs) \[source\]
```

---

**class** `authapp.views.UserListView(**kwargs)` [\[source\]](#)

Bases: `ListAPIView`

API endpoint to list users for the authenticated user based on their role.

Permissions:

- SU: Can view all AU and UU users (excluding frozen).
- AU: Can view UU users they created (excluding frozen).
- UU: No access to this endpoint.

```
get_queryset() \[source\]
```



Get the list of items for this view. This must be an iterable, and may be a queryset. Defaults to using `self.queryset`.

This method should always be used rather than accessing `self.queryset` directly, as `self.queryset` gets evaluated only once, and those results are cached for all subsequent requests.

You may want to override this if you need to provide different querysets depending on the incoming request.

(Eg. return a list of items that is specific to the user)

```
list(request, *args, **kwargs) \[source\]
```

```
permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]
```

```
serializer_class
```

alias of `UserListSerializer`

---

```
class authapp.views.UserProfileView(**kwargs) \[source\]
```

Bases: `RetrieveUpdateAPIView`

API endpoint to retrieve or update the currently authenticated user's profile.

**Permissions:**

- Must be authenticated.

**Actions:**

- GET: Retrieve user's profile information.
- PUT / PATCH: Update user's profile fields.

**Notes**

- Operates on the requesting user (`request.user`) rather than any arbitrary user.

```
get_object() \[source\]
```

Returns the object the view is displaying.

You may want to override this if you need to provide non-standard queryset lookups. Eg if objects are referenced using multiple keyword arguments in the url conf.

```
permission_classes= [<class 'rest_framework.permissions.IsAuthenticated'>]
```

```
serializer_class
```

alias of `UserProfileSerializer`

---

`class authapp.views.VerifyOTPView(**kwargs)` [\[source\]](#)

Bases: `APIView`

Verifies a one-time password (OTP) for a user's password reset.

This view checks if the provided OTP matches the cached OTP, ensures it hasn't already been used, and marks it as verified for subsequent password reset actions.

OTP verification is valid for 10 minutes after successful verification.

`permission_classes=` [`<class 'rest_framework.permissions.AllowAny'>`]

`post(request)` [\[source\]](#)

## Module contents